> **What this is:** nine real cases from this project, compiled from the live code and the project's own **Docs/** folder on 4 Oct 2026. No invented dialogs and no fake logs — every case below traces to a file, a migration or a documented fix, and case 9 shows a live code-review exchange with its actual prompt. Format: **symptom → investigation → root cause → fix → lesson**.

0 How this project is tested

TESTING GATES

- **The build gate.** *vite build* must pass before anything ships — the production build is the fastest full-project check, and it caught the broken JSX in case 6.
- **Component tests.** A vitest setup in both front ends (check vitest.config.ts), with tests such as *handover/CheckIn.test.tsx* mounting real pages and asserting behaviour.
- **API checks as collections.** *API/testers/*.rest* — auth-project, project, register, workshop and maintenance requests you can fire in VS Code's REST client against :5000 with a bearer token.
- **One-command health check.** *check-studio.ps1* reports the three containers, fetches the web app and confirms its JS bundle, then logs in through the API.
- **Data audit.** *docker exec studio_booking_api node audit-damage-evidence.js* — walks every damage record and fails (exit code 1) if a record with a photo resolves to nothing.
- **Migration discipline.** database/init.sql is numbered 000–026; a fresh volume rebuilds the same database every time, so "works on my machine" has one answer: recreate it.

1 The case files

NINE REAL CASES

CASE 1 The date picker that opened behind the map

Symptom. Clicking “All dates” on the Shooting locations page appeared to do nothing — the calendar couldn’t be clicked at all.

Investigation. The popover was rendering *behind* the map.

Root cause. Leaflet’s internal panes paint up to z-index 800, and the map container had no stacking context of its own, while the popover sits at z-index 50 — so every overlay the app opened ended up under the map.

Fix. One line — *.leaflet-container { z-index: 0 }* in index.css gives the map its own stacking context, so calendars, dropdowns and dialogs now always sit above every map in the app.

Lesson. z-index fights aren’t won by raising the popover’s number — they’re won by containing the layer that misbehaves.

CASE 2 The photo that showed on one page but not another

Symptom. A damage photo was visible on the tracked-equipment page, but the damage log, the history dialog and the printed report showed nothing.

Investigation. Two different stores turned out to be involved.

Root cause. Angle/evidence photos live in *equipment_condition_images*, while damage records (damagereports, maintenancelogs) carry a single *imageurl* column. Uploads always wrote the first store and only *best-effort* linked the second — so a record created at another time stayed empty while the file sat safely in /uploads.

Fix (three layers). Write time: the upload endpoints now stamp the item's newest damage report and maintenance log (plus the exact logId/reportId when the screen sends them). **Read time:** one shared rule in damageEvidence.js — the record's own imageurl first, then the newest evidence photo — feeds every damage screen and the PDF. **Verification:** audit-damage-evidence.js fails if any record's photo URL resolves to nothing, and a safe, re-runnable SQL backfill filled the old NULLs.

Lesson. When one fact lives in two tables, there must be a single resolution rule — plus an audit that proves it.

CASE 3 "You do not have permission" — with the buttons right there

Symptom. Staff could see the Equipment approvals buttons, but every action returned "you do not have permission".

Investigation. The page sits in the staff sidebar; the endpoints behind it were admin-only.

Root cause. The UI and the API disagreed about who may do what — the classic two-layer permission drift.

Fix. The routes now accept Studio Staff as well as Admin (the supervision guards), while the decision itself stays stricter — staff propose, admin confirms — matching how the page was designed.

Lesson. "The button exists" and "the action is allowed" are two separate questions. When they disagree, one of the layers is wrong — find out which.

CASE 4 The kit that said "not enough" even though nobody had it

Symptom. Booking the same kit bag for a different day triggered availability complaints — even though the calendar already prevents overlapping bookings and the kit was not out.

Investigation. Kits were being treated as loose stock (a quantity) instead of as a physical bag with dates.

Root cause. One availability model trying to serve two different things.

Fix. Split the model — **kit bags are booked as a unit against the calendar**: their booked windows (from /api/bookings/kit-bookings) block only overlapping dates, and returned / rejected / cancelled bookings are ignored. Individual **extras keep per-unit counters**. Migrations 018–021 separate kit bags from extras and clean up the legacy counters.

Lesson. If a resource is borrowed as a whole object, its availability is *dates*, not quantity. Model it the way it is actually used.

CASE 5 The numbers that lied twice

5a — the 1970 date. A maintenance row with no logged date printed "01 Jan 1970", because *new Date(undefined)* walks straight to the Unix epoch. Fixed by showing a dash when there is no date.

5b — spelling drift. Older rows stored conditions as "Good" / "good" / "Fair"; comparisons now normalise to lower case so mixed-spelling history keeps working.

Lesson. Display bugs often expose data you didn't know you had. Handle the *absence* of a value, not just the value.

CASE 6 The page that went blank after a big edit

Symptom. After a large UI refactor, /app/statistics rendered blank.

Investigation. The production build finally failed with "Unterminated regular expression", pointing at a JSX fragment line.

Root cause. Scripted edits had left orphaned fragment tags and mixed indentation — the component's structure was corrupted without anyone hand-editing it.

Fix. Re-read the exact file, repaired the JSX structure, build passed.

Lesson. After bulk or scripted edits, run the build *before* opening the browser. The blank screen is the symptom; the parser is the truth.

CASE 7 The calendar clinic — when every day looks free

Symptom. A booking calendar renders, but every day looks available (or the grid looks empty).

Checklist (from the project's calendar walkthrough):

- **401 vs 403.** 401 = no/expired token in localStorage; 403 = the token is fine but requireRole does not include your role.
- **The istracked flag.** The fleet query only includes models flagged istracked = true.
- **Null windows.** Booking lines with both pickup and return NULL are skipped on purpose.
- **Statuses.** Returned / rejected / cancelled bookings are ignored by the kit query.
- **Month paging.** Booking history refetches per month — check the from/to it sends.
- **Off-by-one.** Postgres returns timestamps; the grid normalises with startOfDay() on the client — compare the raw API string with the tile it paints.

Lesson. Calendars fail in the data, not in the drawing. Check what the API returned before touching the grid.

CASE 8 Open case — the blank Equipment page on the phone

Symptom. The Equipment page on the Android app showed a blank white screen on the phone.

Status. *Open.* The debugging session (26 Aug) traced the mobile flow but was interrupted before a fix was confirmed.

How to approach it: (1) confirm the phone is on the hotspot and */api/health* answers on the API's LAN address; (2) check whether the page's data request returns 200 on the phone (compare with the laptop); (3) verify the VITE_API_URL baked into the APK matches the machine's current address; (4) then investigate rendering.

Lesson. An open case, documented honestly, is still useful engineering — it tells the next person exactly where to start.

CASE 9 The module with no bouncer — a live review of the project API

projectbooking find something

You · 4 Oct 2026 — the request, verbatim

✳ **Claude — opened the real files, then reported**

Read *project.controller.js* (1,045 lines), *projects.routes.js*, the schema and migrations, and the booking flow that consumes projects — then reported real defects, each one verifiable in the files. Condensed below; every reference is exact:

- **Anyone can approve any project — logged out.** *projects.routes.js:37* patches a project with no *verifyToken*, and the handler accepts *status* with no owner check at all (*project.controller.js:223–307*). One request sets "approved" — which unlocks bookings, because *createBooking* trusts that single word (*booking.controller.js:136*).
- **Anyone can remove any team member.** *projects.routes.js:58* — no token and no creator check; the handler only verifies the row exists before deleting (*project.controller.js:873–910*).
- **Invitee names and emails readable by anyone.** *projects.routes.js:46* has no token — and inside, the "is the requester the project creator?" check is a comment with no code under it (*project.controller.js:534–536*).
- **A rejected invite can flip back to accepted.** The pending check runs outside the transaction, and the accept update has no *AND status='pending'* guard (*project.controller.js:671 vs 717–722*). The reject path does guard; accept doesn't.
- **Two writes, no transaction — and a colliding ID scheme.** *createProject* inserts the project row and the team row separately (*project.controller.js:88–114*); *applicationid* uses the last four digits of a timestamp (*76*), so two projects created 10 seconds apart share an ID — and no migration 000–026 puts a UNIQUE constraint on *projectcode* or *applicationid*.
- **Counters that disagree with the list.** *countProjects* counts only projects you created (*13–38*); *getMyProjects* also counts team membership (*940–941*) — the dashboard can show 0 while the project list shows rows.

Also flagged in the same pass: *coursecode* stores a course name (up to 200 characters) in a VARCHAR(50) column; the team endpoint silently hides members without workshop attendance; the *test-auth* debug route is still mounted; and raw database errors ride along on 500 responses. Honest calibration, from the same review: every query is parameterised (no SQL injection found), *acceptInvite* shows the right transaction pattern, and *bookings.routes.js* guards every one of its routes — the projects router is the outlier, which is exactly why these holes are worth a case file.

Lesson. A guard has two layers — the route (who may call) and the handler (who may touch which row) — and one missing layer can silently defeat a correct guard in another file. And the starting prompt was five words: the review worked because it opened the real code, not because the prompt was clever.

Compiled 4 Oct 2026 from the live code (Team22-API · Team22-Main · Team22-Mobile) and the project's own Docs/ folder. Companion documents: the AI Usage Log (16–26 Aug 2026 sessions) and the System Walkthrough (Web · Mobile · API).